SESSION HIJACKING

Module 11

AVINASH KUMAR

Table of Contents

1. **Learning Objectives**
    - **Describe session hijacking concepts**
    - **Perform application-level session hijacking**
    - **Perform network-level session hijacking**
    - **Use different session hijacking tools**
    - **Apply session hijacking countermeasures**
2. **Session Hijacking Concept**
    - **Definition and overview**
    - **Role of session IDs (cookies, URLs, headers)**
3. **What is Session Hijacking?**
    - **Session tokens and authentication process**
    - **Exploiting weaknesses in token generation**
    - **Impersonation via stolen session tokens**
4. **Why Is Session Hijacking Successful?**
    - **Absence of account lockout/monitoring**
    - **Weak or short session-ID generation**
    - **Insecure handling of session IDs**
5. **Session Hijacking Process**
    - **Impersonation of legitimate users**
    - **Redirection of traffic**
    - **Long-term unauthorized access**
6. **Spoofing vs. Hijacking**
    - **Blind session hijacking**
    - **TCP sequence numbers and prediction**
7. **Application-Level Session Hijacking**
    - **Stealing session IDs**

AVINASH KUMAR

- **Guessing session IDs**
- **Brute-forcing session IDs**

8. **Compromising Session IDs**
    - **Using sniffing**
    - **Man-in-the-Browser (MitB) attack**
        - **Steps involved in MitB attack**
    - **Session fixation attack**
        - **Techniques: URL parameters, hidden fields, cookies**
9. **Cookies Editor Tool**
    - **Functions and features**
    - **Practical demonstration (Chrome → Edge login bypass)**
10. **Session Hijacking Using Proxy Servers**
    - **Malicious redirection**
    - **Attacker as proxy between victim and server**
11. **Burp Suite Concept**
    - **Overview and purpose**
    - **Core components**
12. **Bettercap Tool**
    - **Overview**
    - **ARP poisoning**
13. **Session Hijacking Countermeasures**
    - **Detection challenges and indicators**
    - **Protecting against session hijacking**
        - **Best practices for organizations and users**
14. **Web Development Guidelines to Prevent Session Hijacking**
    - **Secure session management practices**
    - **Encryption and SSL/TLS**

AVINASH KUMAR

- **Session expiration and regeneration**
- **Cookie attributes (HTTPOnly, Secure flag)**
- **Request origin verification**

15. Module Summary

- **Recap of concepts, tools, detection, and prevention**

AVINASH KUMAR

Learning Objectives

Session hijacking allows attackers to take over an active session by bypassing the authentication process. Thereafter, they can perform any action on the hijacked system.

At the end of this module, you will be able to do the following:

- Describe session hijacking concepts
- Perform application-level session hijacking
- Perform network-level session hijacking
- Use different session hijacking tools
- Apply session hijacking countermeasures

AVINASH KUMAR

Session Hijacking Concept

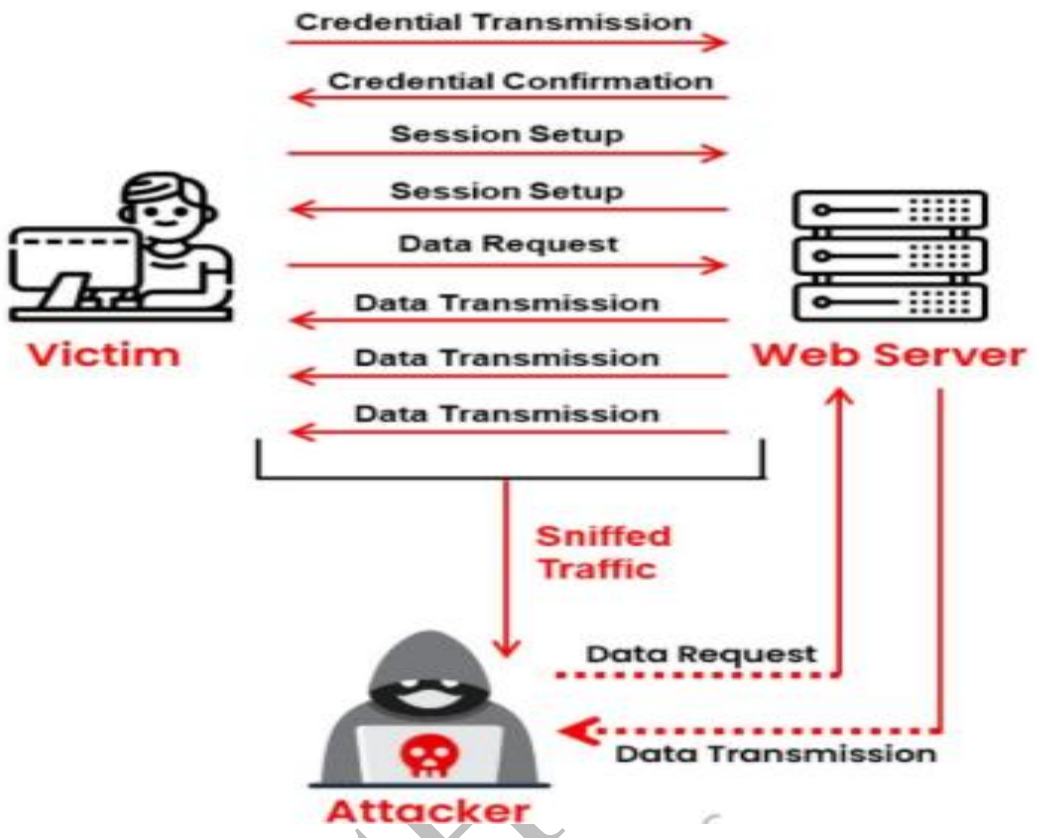
Session hijacking is a cyberattack in which an attacker takes control of an active and authenticated session between a user and a system by stealing or predicting the session ID. Instead of breaking the login credentials, the attacker bypasses the authentication process and impersonates the legitimate user. Session IDs are usually stored in cookies, URLs, or HTTP headers, and if these are exposed through insecure networks, XSS attacks, or man-in-the-middle attacks, they can be exploited. Once the session is hijacked, the attacker can perform any action that the authorized user is allowed to do, such as accessing sensitive data, modifying information, or carrying out fraudulent activities.

What is Session Hijacking?

Session hijacking is a cyberattack in which an attacker takes control of an active communication session between a web client and a web server. After a user successfully authenticates, the server generates a unique session identification token to manage and distinguish multiple user sessions. These tokens are expected to be randomly generated and securely transmitted. However, if an attacker exploits weaknesses in the session-token generation process or its security controls, they can steal or guess a valid session ID.

Since authentication usually occurs only at the beginning of a TCP session, the attacker can misuse the stolen session token to impersonate a legitimate user while the session is still active. As a result, the web server treats the attacker's requests as authorized, enabling activities such as identity theft, information theft, and fraud by sniffing or manipulating ongoing TCP traffic without the user's knowledge.

AVINASH KUMAR



Why Is Session Hijacking Successful?

Session hijacking is successful due to several security weaknesses in web applications. One major reason is the absence of account lockout or monitoring for invalid session IDs. When a website does not restrict repeated attempts with incorrect session identifiers, an attacker can continuously try different session IDs embedded in URLs until a valid one is found. This method is essentially a brute-force attack, and because the server does not generate alerts or warnings during these attempts, the attacker can eventually identify an active session.

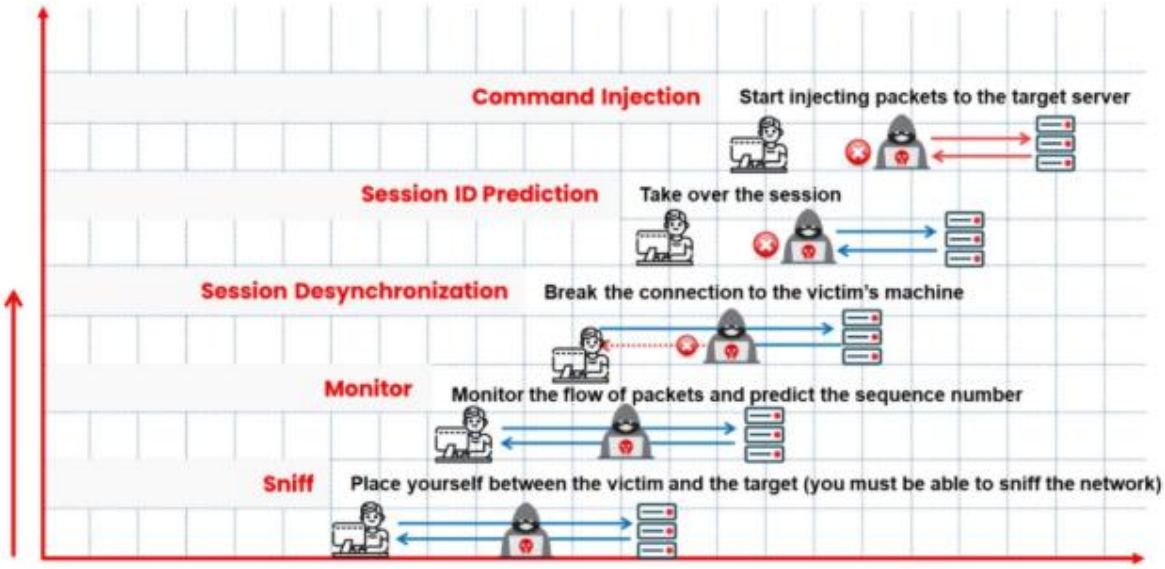
Another reason is the use of a weak session-ID generation algorithm or short session IDs. Many websites generate session identifiers using predictable values such as time stamps or IP addresses. By observing request patterns and session sequences, attackers can reduce the range of possible values and successfully guess a valid session ID. Even when stronger algorithms are used, short session IDs are still vulnerable because they can be cracked more easily.

Finally, insecure handling of session IDs greatly increases the risk of session hijacking. Attackers can trick users into visiting malicious websites that capture stored session information.

AVINASH

Session Hijacking Process

For an attacker, it is often easier to gain access to a system by impersonating a legitimate user rather than attempting a direct intrusion. In session hijacking, the attacker locates an already authenticated and active user session and then takes control of it. Once the session is hijacked, the attacker can remain connected for long periods without raising suspicion. During this time, network traffic meant for the legitimate user's IP address is redirected to the attacker's system. This allows the attacker to monitor communications, install backdoors, and obtain further unauthorized access to the system and its resources.

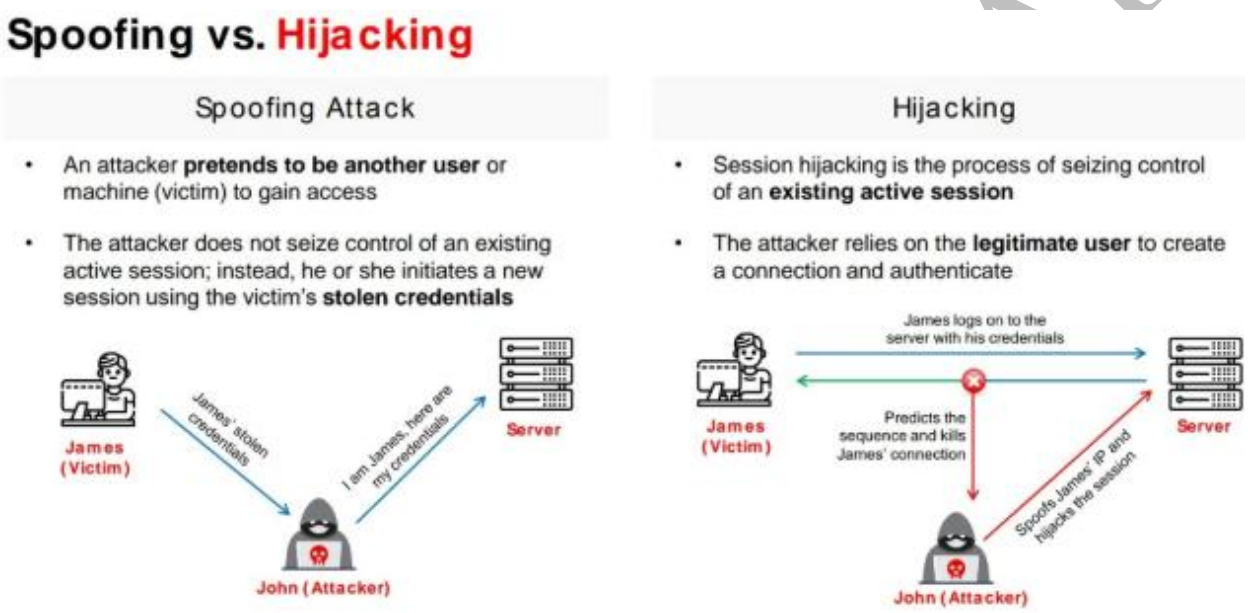


Spoofing vs. Hijacking

In **blind session hijacking**, an attacker forges a connection that appears to originate from a legitimate host by predicting the **TCP sequence numbers** used in an active session. To understand this attack, it is essential to know how TCP sequence numbers function. These numbers are unique for each byte transferred during a TCP session and are responsible for ensuring proper data flow and integrity. During the TCP three-way handshake, each side announces an **Initial Sequence Number (ISN)**, from which subsequent bytes are numbered sequentially. ISNs are not reset to zero for every session, making them a potential target if poorly randomized.

Blind session hijacking depends entirely on the attacker's ability to **guess or predict these sequence numbers**. Because the attacker is spoofing a trusted host located on

a different network, they cannot see the response packets, as no return path exists to their real IP address. Additionally, techniques such as **ARP cache poisoning** cannot be used across the Internet because routers do not forward ARP broadcasts. Since the attacker cannot observe the server's replies, they must accurately anticipate the expected responses and suppress TCP reset (RST) packets that could terminate the connection. By successfully predicting the correct sequence numbers, the attacker takes control of the communication session. This technique is particularly effective when exploiting **trust relationships between systems or users and remote machines**.



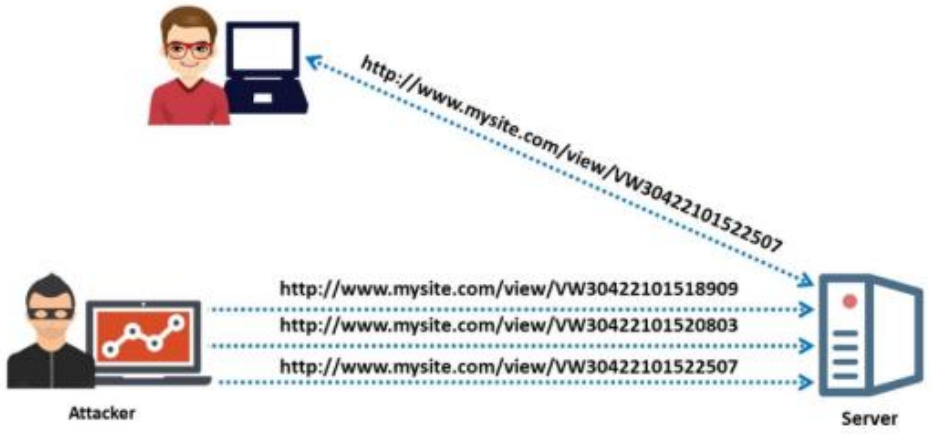
Application-Level Session Hijacking

Application-level session hijacking involves compromising a user's session by stealing or predicting a valid **session token**, which allows an attacker to gain unauthorized access to a web server or to establish a fraudulent session. This type of hijacking primarily targets **HTTP-based sessions**. In many cases, application-level hijacking occurs alongside network-level session hijacking, because access to network traffic often provides attackers with sufficient information to exploit application sessions.

To take control of an active user session, attackers use techniques such as **stealing, guessing, or brute-forcing session IDs** while the session is still active.

- **Stealing:** Attackers may obtain session IDs through physical access to a system, such as by retrieving files or memory data from a user's device or the server. They can also intercept session tokens by sniffing network traffic using packet-capture tools, extracting session IDs directly from client–server communications.
- **Guessing:** In this method, attackers analyze session variables and attempt to predict valid session IDs. Since the possible range of session ID values is often limited, guessing is effective mainly when the server uses weak, predictable, or poorly designed session-ID generation mechanisms.
- **Brute forcing:** Brute-force attacks involve systematically trying all possible combinations of session ID values until a valid one is discovered. With high-speed connections such as DSL, attackers can generate and test a large number of session IDs in a short time, increasing the likelihood of success.

This combination of weaknesses makes application-level session hijacking a significant threat when proper session management controls are not in place.

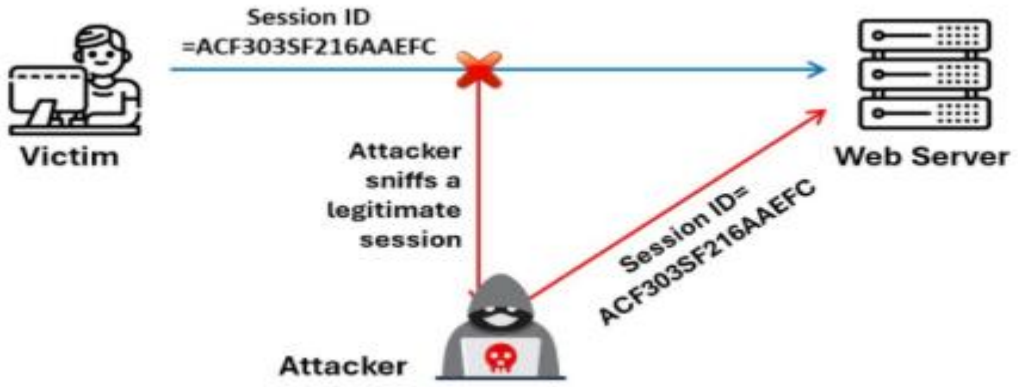


AVINASH KUMAR

Compromising Session IDs Using Sniffing

A web server recognizes a user's connection by assigning a **unique session ID**, also called a session token. After a client successfully logs in, the server sends this token to the user's browser. The session token is typically a variable-length string and may be transmitted in different ways, such as within HTTP cookies, embedded in URLs, or included in the body or headers of HTTP requests.

Attackers exploit this mechanism by using **packet-sniffing tools** to capture HTTP traffic exchanged between the victim and the web server. By analyzing the intercepted packets, attackers can extract sensitive information, including **session IDs and login credentials**. Once a valid session ID is obtained, the attacker impersonates the legitimate user by submitting the stolen token to the web server, often before the real user does. As a result, the server accepts the attacker's requests as authenticated, allowing unauthorized access and enabling the attacker to take control of an active, legitimate session.



Compromising Session IDs Using Man-in-the-Browser / Manipulator-in-the-Browser Attack

A **man-in-the-browser (MitB)** or **manipulator-in-the-browser** attack is a variation of a man-in-the-middle (MITM) attack. Unlike traditional MITM attacks that intercept traffic between two communicating systems, a MitB attack operates **inside the victim's browser** by using a **Trojan horse malware**. This Trojan intercepts and alters communication between the browser and its built-in security mechanisms or libraries.

In this attack, the malware is placed between the browser and its security controls, allowing it to **modify web pages, alter transaction data, or insert fraudulent transactions**. These malicious activities remain completely invisible to both the user and the web application. The primary goal of a MitB attack is **financial fraud**, particularly targeting online banking transactions.

A key characteristic of man-in-the-browser attacks is that they can succeed even when strong security measures such as **SSL/TLS, public key infrastructure (PKI), and two-factor authentication** are in place. Since the attack occurs after authentication and within the browser itself, all security controls appear to function normally.

Steps Involved in a Man-in-the-Browser Attack

- The Trojan first infects the victim's system, either at the operating system or application level.
- The malware installs malicious code, usually in the form of **browser extension files**, and stores them in the browser configuration.
- When the user restarts the browser, the malicious extensions are automatically loaded.
- These extensions register handlers that monitor every webpage visit.
- When a webpage is accessed, the extension compares the URL with a predefined list of targeted websites.
- The user logs in securely to the website, after which the Trojan silently intercepts and manipulates the session or transactions.

AVINASH KUMAR

Compromising Session IDs Using Session Fixation (Rephrased)

Web session security mechanisms are designed to protect session IDs from being intercepted, guessed, or brute-forced after they are issued by a web server to an authenticated user. However, these protections often overlook a critical risk: an attacker can **assign a session ID to the user's browser in advance** and force the browser to reuse it. This technique is known as a **session fixation attack**, in which the attacker predefines the session ID instead of allowing the server to generate a fresh one at login.

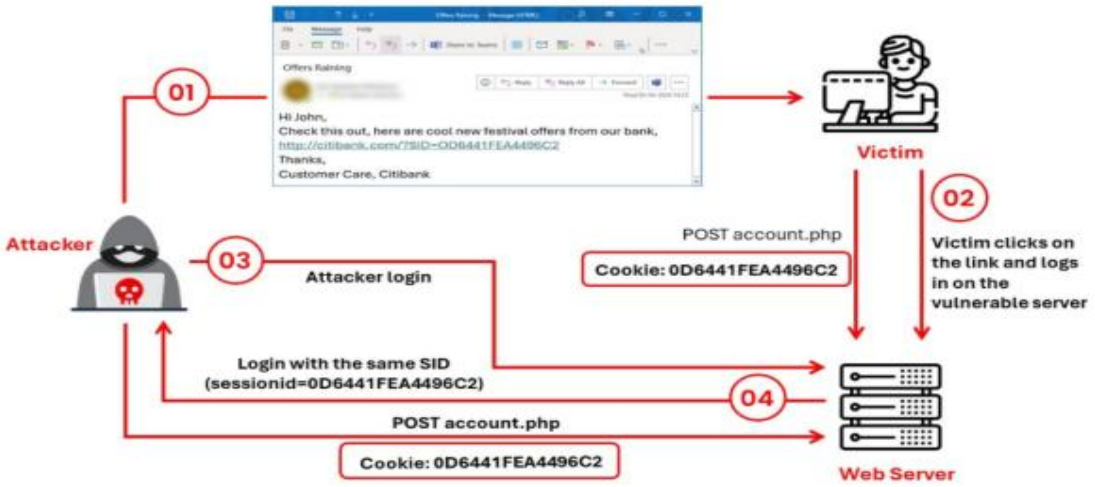
In a session fixation attack, the attacker exploits weaknesses in **session ID management** within a web application. Some applications allow users to authenticate using an already existing session ID rather than issuing a new one after successful login. The attacker first obtains or creates a valid session ID and then tricks the victim into using that session.

Although session fixation is a form of session hijacking, it differs from traditional hijacking attacks. Instead of stealing the session ID after the user has logged in, the attacker **sets the session ID before authentication occurs**, making the attack possible even before the session becomes active.

Attackers may carry out session fixation using several techniques, including:

- Embedding the **session token in a URL parameter**
- Injecting the **session token into hidden form fields**
- Forcing a **session ID through browser cookies**

These methods allow the attacker to control the session identifier that the victim unknowingly uses.



AVINASH KUMAR

Cookies Editor Tool

A **Cookies Editor** is a **browser-based utility or extension** used to **view, add, modify, delete, export, and import HTTP cookies** stored by websites in a user's browser. Cookies hold session-related data such as **session IDs, authentication tokens, preferences, and tracking values**, making this tool important for both **web development and cybersecurity education**.

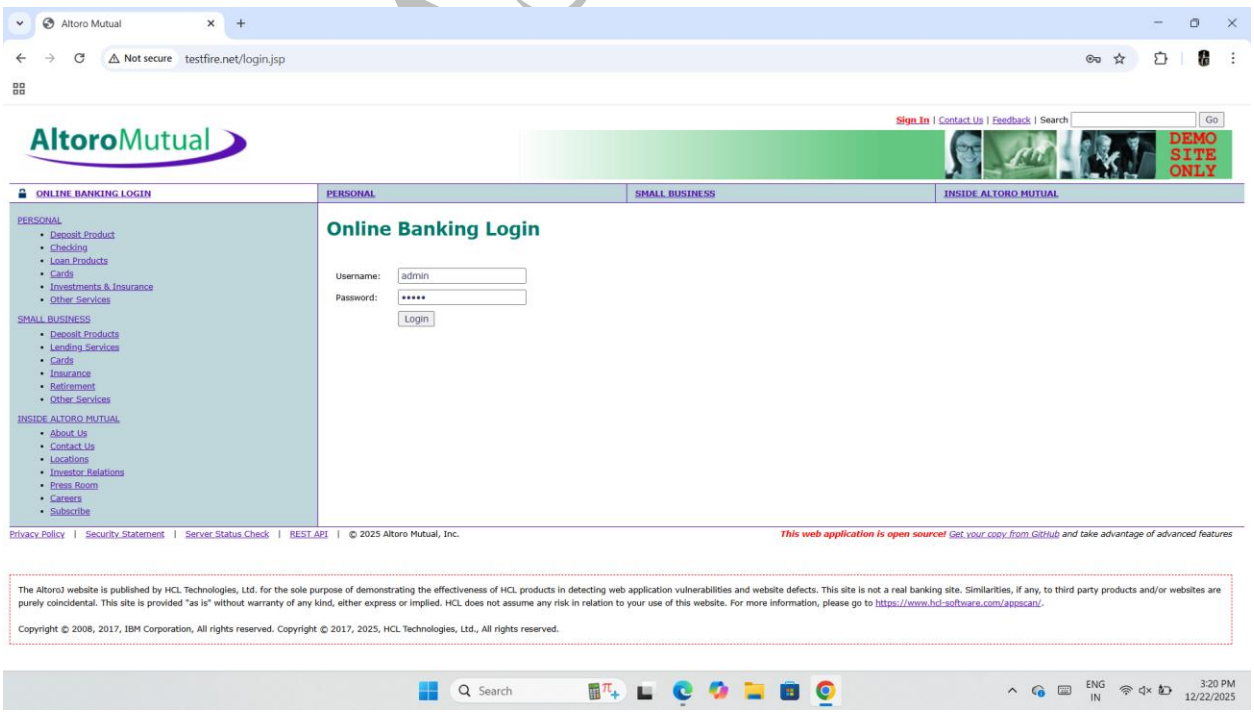
What a Cookies Editor Does

- Displays all cookies for a selected website
- Allows manual **editing of cookie values**
- Enables **deleting or injecting cookies**
- Supports **import/export** of cookies for testing
- Shows attributes like **Name, Value, Domain, Path, Expiry, Secure, HttpOnly, SameSite**

Download cookies editor from website :

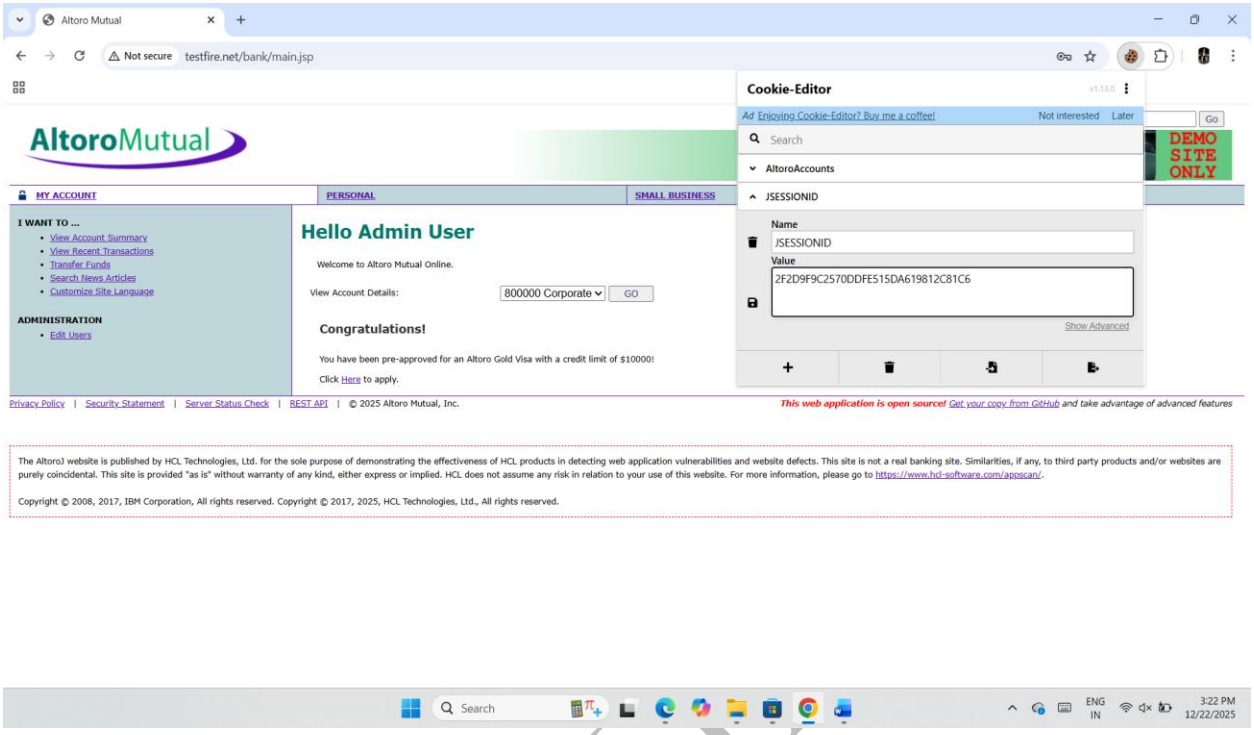
https://chromewebstore.google.com/detail/cookie

Login and visit cookies editor extension from extension

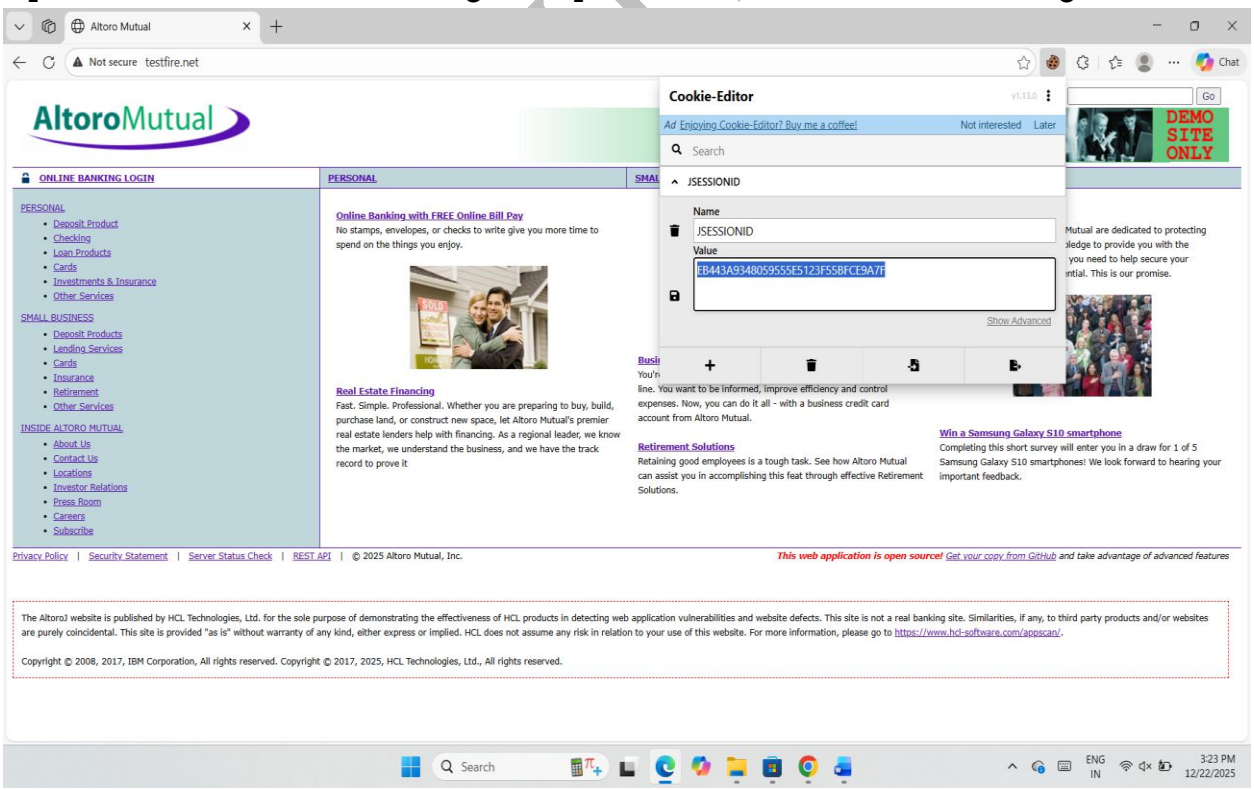


AVINASH KUMAR

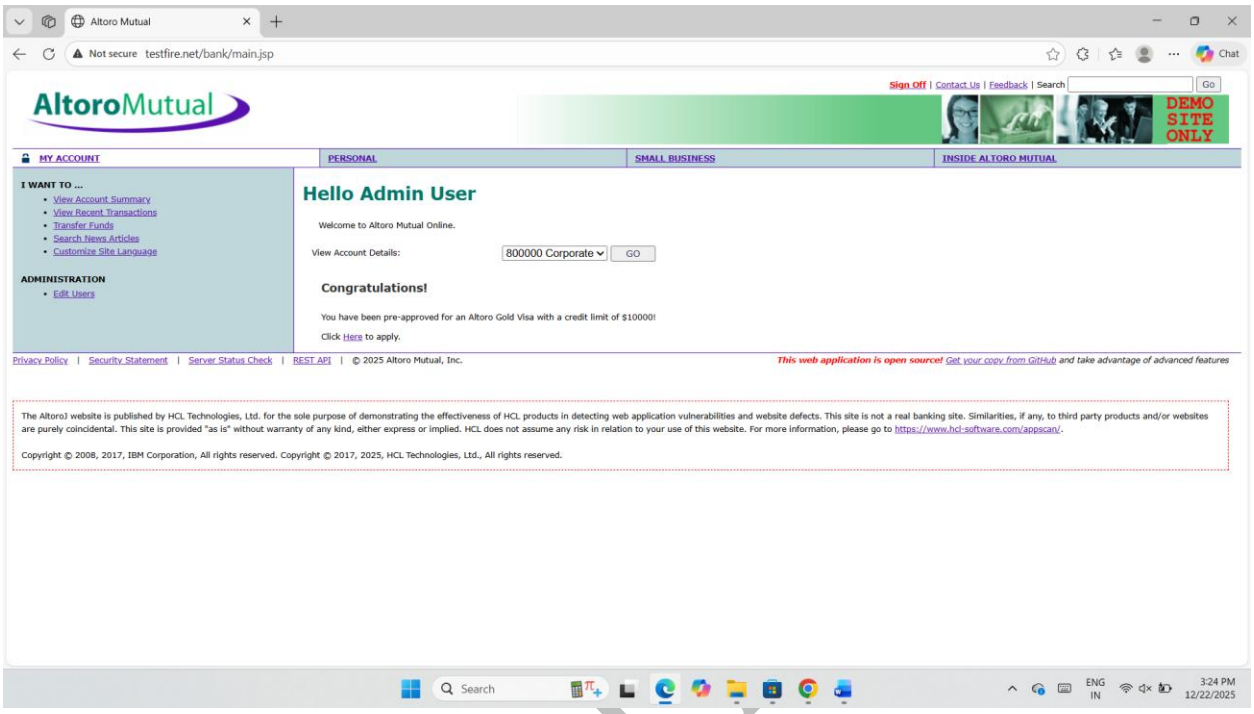
Click and copy the extension value



Open other browser such as edge and paste the value from crome to edge.



refresh the page as you can see below the same page has login without the credentials



Session Hijacking Using Proxy Servers

In session hijacking using proxy servers, an attacker tricks the victim into clicking a **malicious link** that looks legitimate but secretly redirects the user to the attacker's server. The attacker's server then forwards the victim's request to the real website, acting as an **intermediate proxy** between the user and the legitimate server. Throughout the communication, the attacker relays responses back to the victim while silently monitoring the exchange. By functioning as a proxy for the entire session, the attacker is able to **capture sensitive session information**, such as session IDs or authentication tokens, during the interaction between the user and the genuine web server.

Burp Suite Concept

Burp Suite is a widely used **web application security testing platform** developed by PortSwigger. It acts as an **intercepting proxy** between a user's browser and a web server, allowing security professionals and students to **inspect, modify, and analyze HTTP/HTTPS traffic**. Burp Suite is primarily used for **ethical hacking, penetration testing, and web application vulnerability assessment**.

How Burp Suite Works

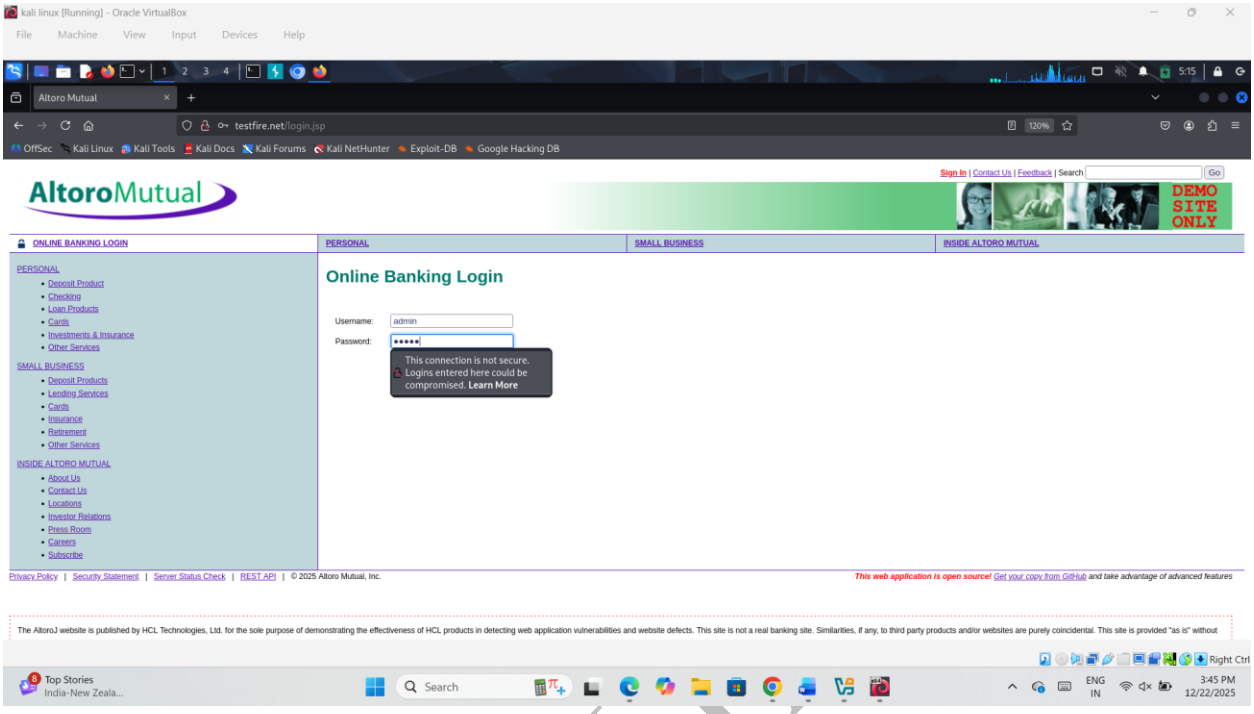
Burp Suite sits between the browser and the target web application. All requests sent by the browser and responses returned by the server pass through Burp, where they can be **intercepted, viewed, edited, or replayed**. This makes it easier to understand how web applications handle input, authentication, sessions, and data.

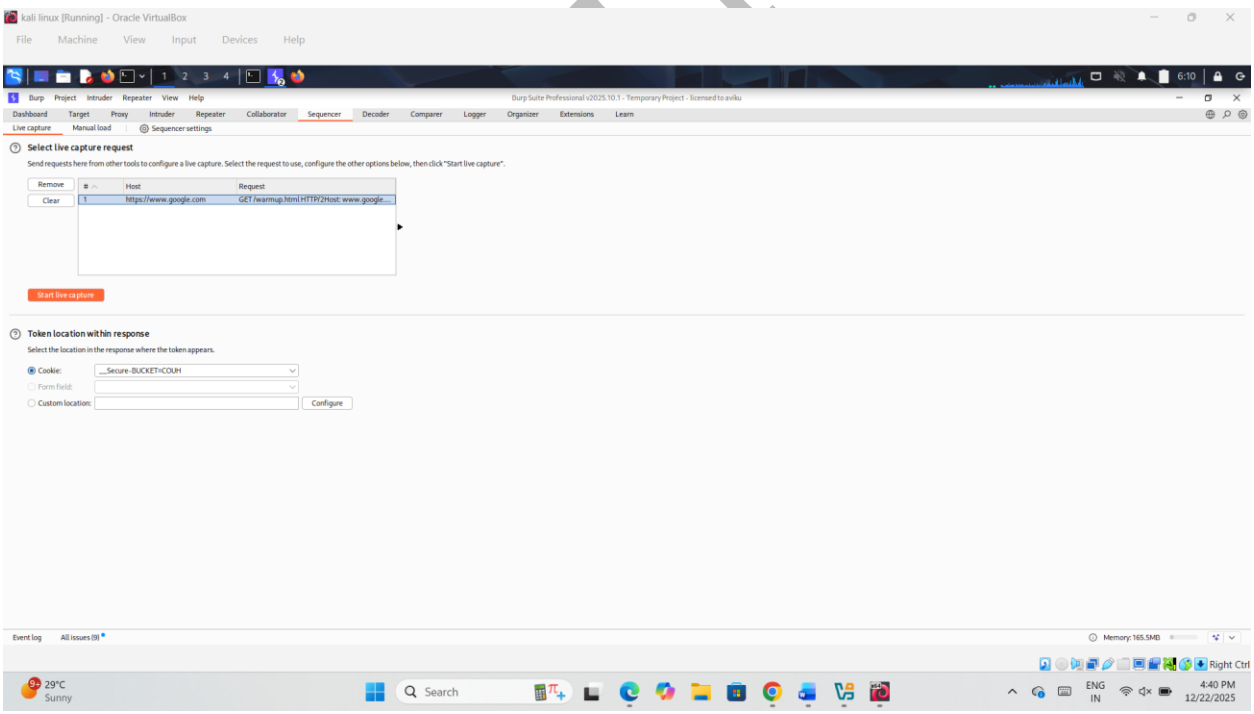
Core Components

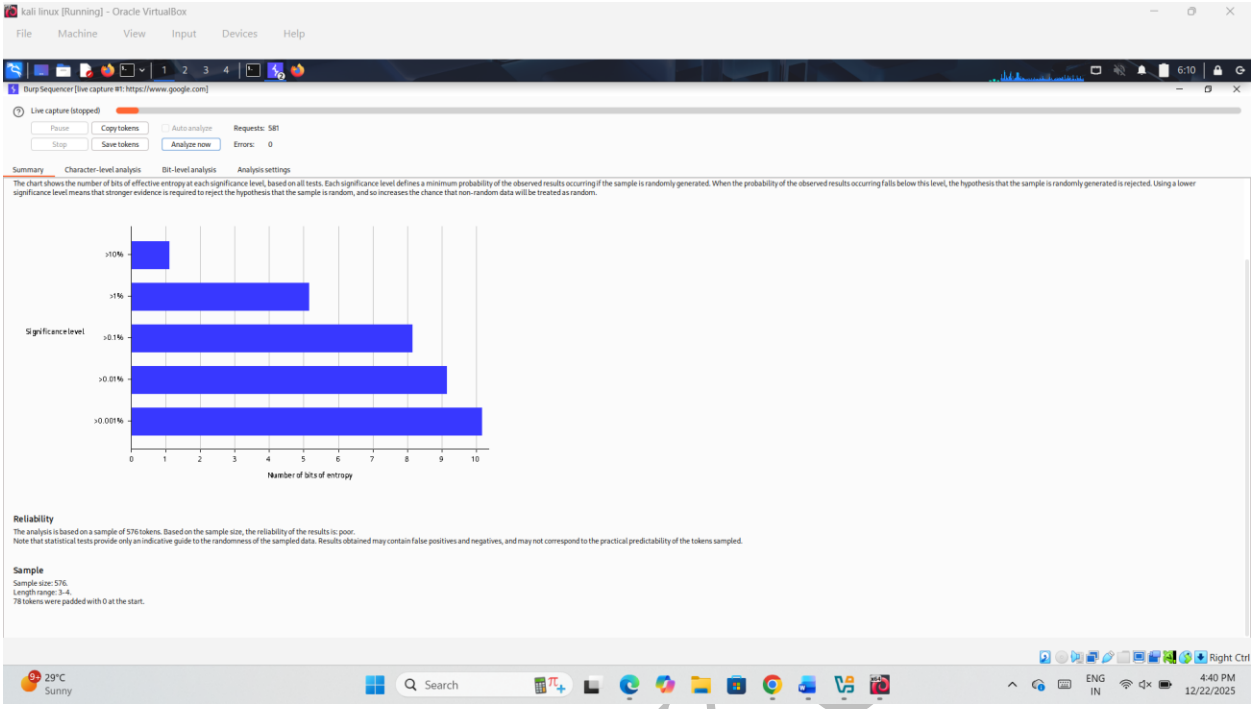
- **Proxy** – Intercepts and modifies HTTP/HTTPS requests and responses
- **Repeater** – Re-sends requests manually to test application behavior
- **Intruder** – Performs automated attacks like brute force and fuzzing (authorized use only)
- **Scanner** – Automatically identifies common vulnerabilities (available in Pro version)
- **Decoder** – Encodes/decodes data (Base64, URL, Hex, etc.)
- **Comparer** – Compares responses to identify differences

AVINASH KUMAR

Using burpsuite







the **Burp Suite → Sequencer** tool, which is used to **analyze the randomness and predictability of session tokens** (such as cookies, session IDs, or CSRF tokens). The analysis helps determine whether a web application is vulnerable to **session hijacking or session prediction attacks**.

1. What Is Being Analyzed

- **Target**: Tokens captured from HTTPS traffic (example shown: google.com)
- **Sample size**: ~576 tokens
- **Token length**: ~34 characters
- **Prefixed bytes**: 7 bytes were predictable at the start

Burp Sequencer evaluates whether the token values are **random enough** to resist guessing or brute-force attacks.

2. Character-Level Analysis (Bar Graph)

- The graph shows **entropy distribution** at different **significance levels**.
- **Entropy** measures randomness (higher = more secure).
- Tokens with **more bits of entropy** require significantly more effort to predict.

Observation:

- Entropy increases as the significance level decreases.

AVINASH K

- This suggests the token has **some randomness**, but not conclusively strong due to sample limitations.

3. Significance Levels Explained

- **High significance level (e.g., >10%)**
  → Easier to reject randomness
- **Low significance level (e.g., <0.01%)**
  → Stronger confidence required to claim non-randomness

Interpretation:
At stricter significance levels, more entropy bits are observed, which is a **positive indicator**, but not definitive.

4. Reliability Warning (Very Important)

Burp clearly states:

"Based on the sample size, the reliability of the results is poor."

This means:

- The **number of collected tokens is too small**
- Results may contain **false positives or false negatives**
- The analysis should be treated as **indicative, not final**

Key Point for Exams & Reports:

Low sample size = low confidence in randomness assessment.

5. Security Interpretation

- Tokens show **partial randomness**
- Presence of **predictable prefix** (7 bytes) is a **potential weakness**
- Small sample size prevents a strong conclusion
- If token generation is weak, attackers could attempt:
  - Session prediction
  - Session fixation
  - Session hijacking

AVINASH KUMAR

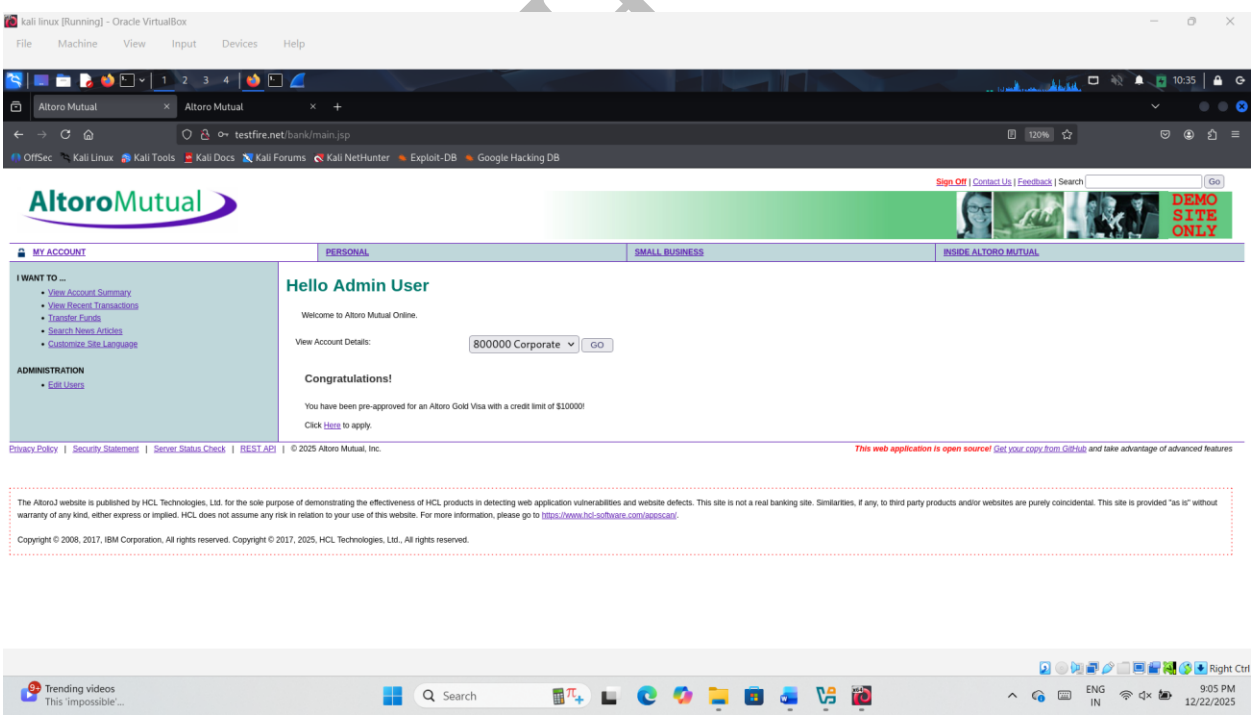
Bettercap Tool

Bettercap is a powerful **network attack and monitoring framework** mainly used in **penetration testing, red teaming, and digital forensics labs**. It is designed to **intercept, manipulate, and analyze network traffic in real time**, especially in **LAN, Wi-Fi, and IoT environments**.

At its core, Bettercap works as a **man-in-the-middle (MITM) platform**. It performs **ARP poisoning** to position itself between a victim and the gateway, allowing it to **sniff packets, modify traffic, inject payloads, and steal sensitive data** such as credentials, sessions, and DNS queries. Unlike older tools, Bettercap uses a **modular architecture**, where individual "caplets" enable or disable specific attacks or monitoring features.

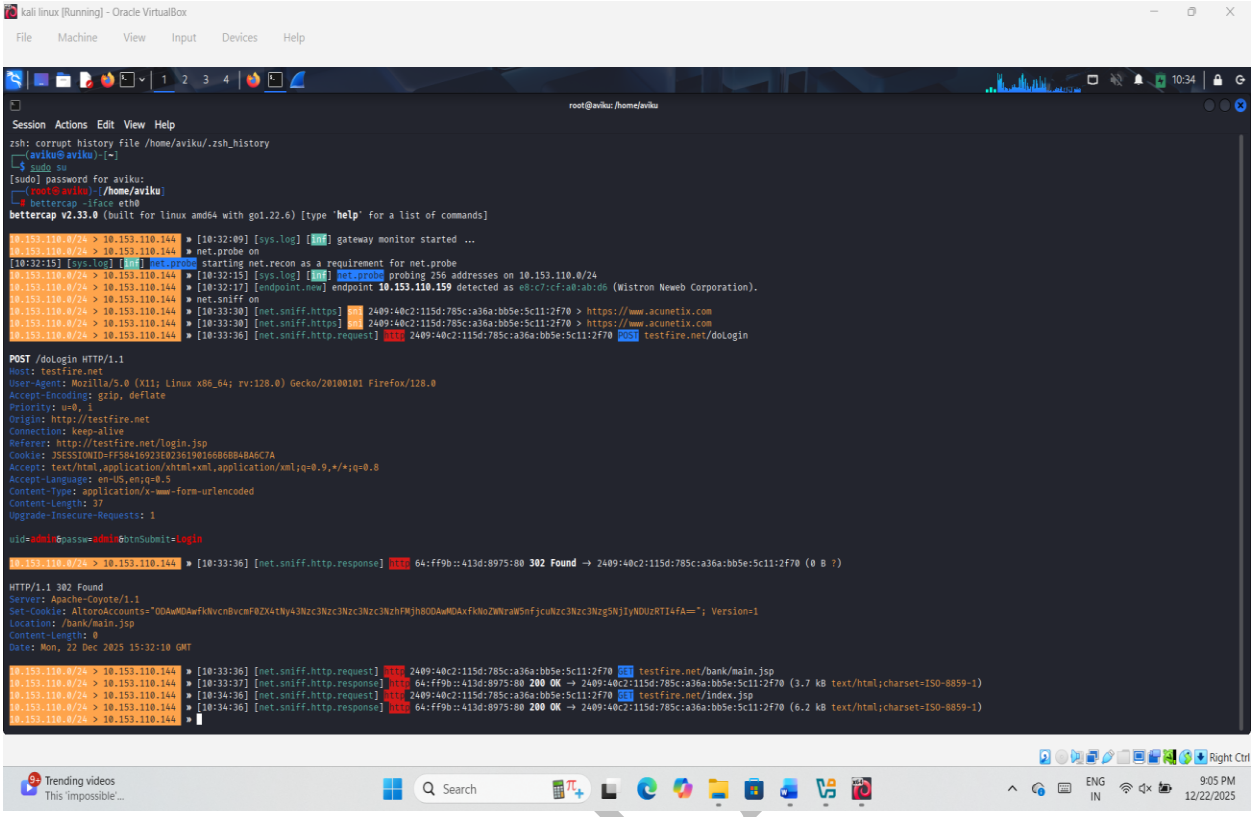
ARP poisoning (also called **ARP spoofing**) is a network attack in which an attacker sends **forged ARP (Address Resolution Protocol) messages** on a local area network to associate their **MAC address** with the **IP address of a legitimate device**, such as the default gateway or another host. As a result, network traffic intended for the legitimate device is **redirected to the attacker**.

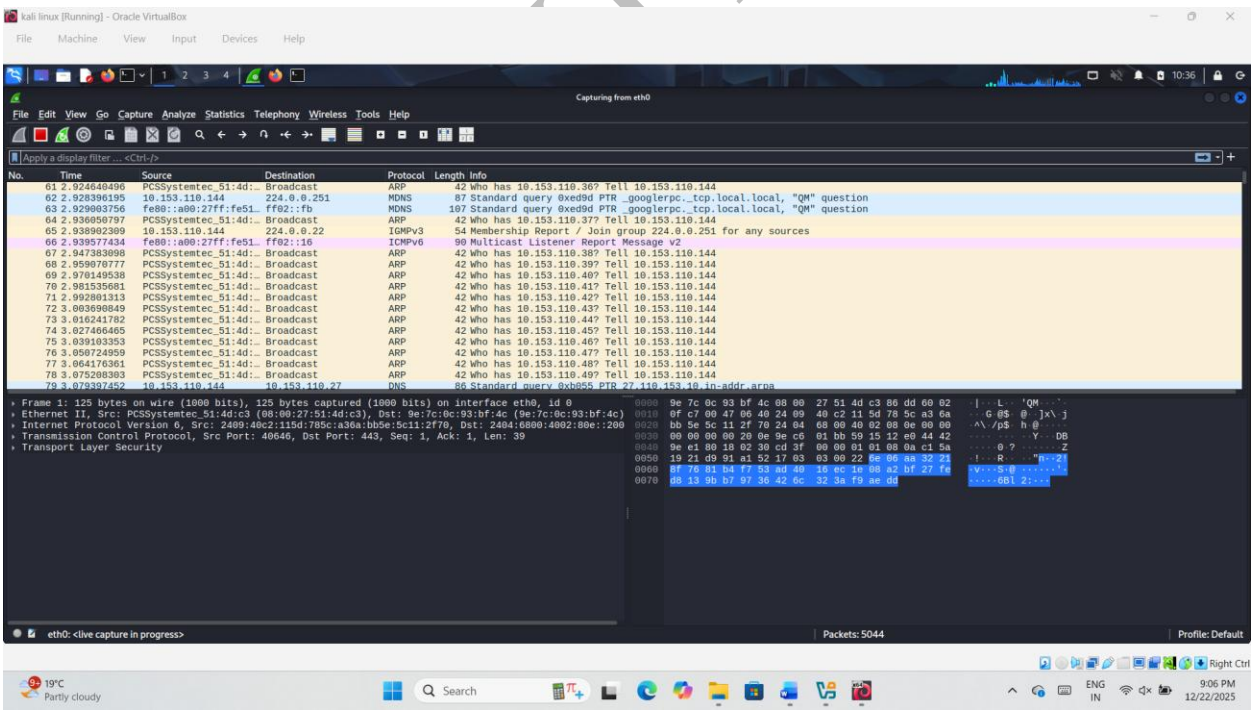
Open altoro mutual fund and login and open terminal and use bettercap cmd



Use cmd net.probe on

net.sniff on


kali linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
root@aviku: /home/aviku
Session Actions Edit View Help
zsh: corrupt history file /home/aviku/.zsh_history
(aviku㉿aviku)-[~]
$ sudo su
[sudo] password for aviku:
(root㉿aviku)-[/home/aviku]
bettercap -iface eth0
bettercap v2.33.0 (built for linux amd64 with go1.22.6) [type 'help' for a list of commands]
10.153.110.0/24 > 10.153.110.144 » [10:32:09] [sys.log] [inf] gateway monitor started ...
10.153.110.0/24 > 10.153.110.144 » net.probe on
[10:32:15] [sys.log] [inf] net.probe starting net.recon as a requirement for net.probe
10.153.110.0/24 > 10.153.110.144 » [10:32:15] [sys.log] [inf] net.probe probing 256 addresses on 10.153.110.0/24
10.153.110.0/24 > 10.153.110.144 » [10:32:17] [endpoint.new] endpoint 10.153.110.159 detected as e8:c7:cf:a0:ab:d6 (Wistron Neweb Corporation).
10.153.110.0/24 > 10.153.110.144 » net.sniff on
10.153.110.0/24 > 10.153.110.144 » [10:33:30] [net.sniff.https] sni 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70 > https://www.acunetix.com
10.153.110.0/24 > 10.153.110.144 » [10:33:30] [net.sniff.https] sni 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70 > https://www.acunetix.com
10.153.110.0/24 > 10.153.110.144 » [10:33:36] [net.sniff.http.request] http 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70 POST testfire.net/doLogin
POST /doLogin HTTP/1.1
Host: testfire.net
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept-Encoding: gzip, deflate
Priority: u=0, i
Origin: http://testfire.net
Connection: keep-alive
Referer: http://testfire.net/login.jsp
Cookie: JSESSIONID=FF58416923E0236190166B6BB4BA6C7A
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-form-urlencoded
Content-Length: 37
Upgrade-Insecure-Requests: 1
uid=admin&passw=admin&btnSubmit=Login
10.153.110.0/24 > 10.153.110.144 » [10:33:36] [net.sniff.http.response] http 64:ff9b::413d:8975:80 302 Found → 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70 (0 B ?)
HTTP/1.1 302 Found
Server: Apache-Coyote/1.1
Set-Cookie: AltoroAccounts="ODAwMDAwfkNvcnBvcmF0ZX4tNy43Nzc3Nzc3Nzc3NzhFMjh8ODAwMDAxfkNoZWNraW5nfjcuNzc3Nzc3Nzg5NjIyNDUzRTI4fA==”; Version=1
Location: /bank/main.jsp
Content-Length: 0
Date: Mon, 22 Dec 2025 15:32:10 GMT
10.153.110.0/24 > 10.153.110.144 » [10:33:36] [net.sniff.http.request] http 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70 GET testfire.net/bank/main.jsp
10.153.110.0/24 > 10.153.110.144 » [10:33:37] [net.sniff.http.response] http 64:ff9b::413d:8975:80 200 OK → 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70 (3.7 kB text/html;charset=ISO-8859-1)
10.153.110.0/24 > 10.153.110.144 » [10:34:36] [net.sniff.http.request] http 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70 GET testfire.net/index.jsp
10.153.110.0/24 > 10.153.110.144 » [10:34:36] [net.sniff.http.response] http 64:ff9b::413d:8975:80 200 OK → 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70 (6.2 kB text/html;charset=ISO-8859-1)
10.153.110.0/24 > 10.153.110.144 »
Trending videos
This 'impossible'...
Search
ENG IN
9:05 PM
12/22/2025


kali linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
61 2.924640496 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.36? Tell 10.153.110.144
62 2.928390195 10.153.110.144 224.0.0.251 MDNS 87 Standard query 0xed9d PTR _googlerpc._tcp.local.local, "QM" question
63 2.929003756 fe80::a00:27ff:fe51... ff02::fb MDNS 107 Standard query 0xed9d PTR _googlerpc._tcp.local.local, "QM" question
64 2.936050797 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.37? Tell 10.153.110.144
65 2.938902309 10.153.110.144 224.0.0.22 IGMPv3 54 Membership Report / Join group 224.0.0.251 for any sources
66 2.939577434 fe80::a00:27ff:fe51... ff02::16 ICMPv6 90 Multicast Listener Report Message v2
67 2.947383098 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.38? Tell 10.153.110.144
68 2.959070777 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.39? Tell 10.153.110.144
69 2.970149538 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.40? Tell 10.153.110.144
70 2.981535681 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.41? Tell 10.153.110.144
71 2.992801313 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.42? Tell 10.153.110.144
72 3.003690849 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.43? Tell 10.153.110.144
73 3.016241782 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.44? Tell 10.153.110.144
74 3.027466465 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.45? Tell 10.153.110.144
75 3.039103353 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.46? Tell 10.153.110.144
76 3.050724959 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.47? Tell 10.153.110.144
77 3.064176363 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.48? Tell 10.153.110.144
78 3.075208303 PCSSystemtec_51:4d:... Broadcast ARP 42 Who has 10.153.110.49? Tell 10.153.110.144
79 3.079397452 10.153.110.144 10.153.110.27 DNS 86 Standard query 0xb055 PTR 27.110.153.10.in-addr.arpa
Frame 1: 125 bytes on wire (1000 bits), 125 bytes captured (1000 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_51:4d:c3 (08:00:27:51:4d:c3), Dst: 9e:7c:0c:93:bf:4c (9e:7c:0c:93:bf:4c)
Internet Protocol Version 6, Src: 2409:40c2:115d:785c:a36a:bb5e:5c11:2f70, Dst: 2404:6800:4002:80e::200
Transmission Control Protocol, Src Port: 40646, Dst Port: 443, Seq: 1, Ack: 1, Len: 39
Transport Layer Security
eth0: <live capture in progress>
Packets: 5044
Profile: Default
19°C
Partly cloudy
Search
ENG IN
9:06 PM
12/22/2025

Session Hijacking Countermeasures

Session hijacking is a highly dangerous attack because it exposes victims to identity theft, financial fraud, and unauthorized access to sensitive information. Since all TCP/IP–based networks are susceptible to various forms of session hijacking, complete immunity is difficult to achieve. However, adopting security best practices can significantly reduce the risk of such attacks.

This section explains methods for detecting session hijacking, tools used for detection, effective countermeasures to prevent these attacks, and common vulnerabilities that lead to session hijacking along with their preventive solutions, such as the use of IP Security (IPsec).

Detecting session hijacking is particularly challenging because these attacks often remain unnoticed unless the attacker performs activities that cause noticeable damage. Some common indicators of a session hijacking attack include sudden spikes in network traffic that degrade system performance and overloaded servers caused by simultaneous requests from both the legitimate user and the attacker.

AVINASH KUMAR

Protecting Against Session Hijacking

To safeguard against session hijacking, organizations and users should follow these best practices:

- Use **Secure Shell (SSH)** to establish encrypted and secure communication channels.
- Transmit **authentication cookies over HTTPS** to prevent interception.
- Implement a proper **log-out function** so users can safely end their sessions.
- Generate **session IDs** only after a successful login and accept **server-generated IDs exclusively**.
- Ensure all **data in transit is encrypted** and adopt a **defense-in-depth strategy**.
- Use **long, random strings** as session keys for better security.
- Maintain **unique usernames and passwords** for different accounts to reduce compromise risk.
- Educate employees about security best practices and **limit remote access** where possible.
- Implement **session timeouts** to automatically terminate expired sessions.
- Avoid placing **session IDs in URLs or query strings**, which can be easily exposed.
- Upgrade from **hub-based networks to switch-based networks** to minimize ARP spoofing and other hijacking risks.
- Keep both **client-side and server-side security software active and up-to-date**.
- Use **strong authentication mechanisms**, such as Kerberos, or secure connections via **peer-to-peer VPNs**.
- Configure **internal and external spoofing rules on network gateways** to prevent unauthorized access.

AVINASH KUMAR

Web Development Guidelines to Prevent Session Hijacking

Session hijacking often occurs when attackers exploit weaknesses in session management mechanisms. Web developers must prioritize security during development to minimize or eliminate these risks. The following guidelines help protect web applications from session hijacking:

- **Use long, random session keys** that are hard to predict or guess by attackers.
- **Regenerate the session ID after a successful login** to prevent session fixation attacks.
- **Encrypt all data and session keys** exchanged between users and web servers.
- **Implement SSL/TLS** to secure all information transmitted over the network.
- Ensure **sessions expire immediately upon user logout** to prevent reuse.
- Take measures to **prevent network eavesdropping**.
- **Limit the lifespan of sessions** to reduce the window of vulnerability.
- Apply **restrictive cache directives** for all web traffic, such as Cache-Control: no-cache, no-store and Pragma: no-cache headers, or equivalent META tags, especially on sensitive pages.
- Avoid creating **sessions for unauthenticated users** unless absolutely necessary.
- Set the **HTTPOnly attribute** for cookies used as session IDs to prevent client-side access via scripts.
- Use the **Secure flag** for cookies to ensure they are only sent over HTTPS and encrypt them before transmission.
- **Verify the origin of requests** by checking that all actions for a session come from the same IP address and user agent.

AVINASH KUMAR

Module Summary

In this module, we explored the fundamental concepts of session hijacking and examined its various types. We discussed both **application-level** and **network-level** session hijacking attacks in detail and reviewed a range of tools commonly used to carry out these attacks. Additionally, the module covered methods for **detecting, preventing, and defending** against session hijacking, along with an overview of different detection and prevention tools. The discussion concluded with a comprehensive explanation of the **countermeasures** that can be implemented to protect systems and networks from session hijacking attempts by threat actors.

AVINASH KUMAR